



Fundamentos de Algoritmos: Implementación, Complejidad y Estructuras

Algoritmos y Estructuras de Datos

Autor:

Apaza Calizaya Rodrigo Josseph

Docente:

ZANABRIA GALVEZ ALDO HERNAN

Fecha:

05 de mayo de 2026

Índice

1. Introducción a los Algoritmos	2
1.1. Características Principales	2
1.2. Aplicaciones en el Mundo Real	2
2. Análisis de Algoritmos Específicos	2
2.1. Algoritmo 1: Verificación de Palíndromo	2
2.1.1. Pseudocódigo	2
2.1.2. Estructura de Datos y Justificación	3
2.2. Algoritmo 2: Segundo Elemento más Grande	3
2.2.1. Pseudocódigo	3
2.2.2. Estructura de Datos y Justificación	3
2.3. Algoritmo 3: Cálculo de Tarifa	4
2.3.1. Pseudocódigo	4
2.3.2. Estructura de Datos y Justificación	4
3. Análisis de Complejidad Temporal	4
3.1. Metodología de Cálculo de Complejidad	4
4. Optimización y Límites Teóricos	6
4.1. ¿Es posible mejorar la complejidad $O(n)$?	6
4.2. Justificación Técnica	6
5. Conclusión	6

1. Introducción a los Algoritmos

Un **algoritmo** se define como una secuencia finita, ordenada y precisa de pasos o instrucciones destinados a resolver un problema o ejecutar una tarea.

1.1. Características Principales

- **Precisión:** Instrucciones claras sin ambigüedades.
- **Finitud:** Debe terminar en algún punto.
- **Definición:** Ante las mismas entradas, siempre produce el mismo resultado.

1.2. Aplicaciones en el Mundo Real

Los algoritmos no son exclusivos de la informática; se presentan en:

- **Vida Cotidiana:** Recetas de cocina, manuales de montaje y procesos matemáticos manuales.
- **Tecnología:** Motores de búsqueda (Ranking), Sistemas de navegación (Rutas óptimas) y Redes Sociales (Filtros de contenido).

2. Análisis de Algoritmos Específicos

A continuación, se presentan tres algoritmos implementados en C++, su lógica en pseudocódigo y el análisis de su estructura. Los recursos y códigos fuente pueden consultarse en el siguiente repositorio: <https://github.com/RodrigoApaza7/Algoritmos-y-estructuras-de-datos>

2.1. Algoritmo 1: Verificación de Palíndromo

Descripción: Determina si una cadena se lee igual al derecho que al revés, ignorando caracteres no alfanuméricos.

2.1.1. Pseudocódigo

```
1 Funcion EsPalindromo(cadena)
2     cadenaLimpia = Limpiar(cadena)
3     inicio = 0, fin = Longitud(cadenaLimpia) - 1
4     Mientras inicio < fin Hacer:
5         Si cadenaLimpia[inicio] != cadenaLimpia[fin] Entonces
```

```

6         Retornar Falso
7     Fin Si
8     inicio++, fin--
9 Fin Mientras
10    Retornar Verdadero
11 Fin Funcion

```

Listing 1: Lógica de verificación de palíndromo

2.1.2. Estructura de Datos y Justificación

Se utiliza la estructura `std::string` (arreglo dinámico de caracteres). La elección se basa en la necesidad de **acceso aleatorio** ($O(1)$) para comparar caracteres en extremos opuestos de manera eficiente.

2.2. Algoritmo 2: Segundo Elemento más Grande

Descripción: Encuentra el segundo valor máximo en un arreglo de enteros en una sola pasada.

2.2.1. Pseudocódigo

```

1 Inicio
2     Si arr[0] > arr[1] Entonces: may = arr[0], seg = arr[1]
3     Sino: may = arr[1], seg = arr[0]
4     Para i desde 2 hasta n - 1 Hacer:
5         Si arr[i] > may Entonces:
6             seg = may, may = arr[i]
7         Sino Si arr[i] > seg Entonces:
8             seg = arr[i]
9     Fin Para
10 Fin

```

Listing 2: Encontrar el segundo mayor

2.2.2. Estructura de Datos y Justificación

Se utiliza un **Arreglo (Array)**. Es la estructura óptima para recorridos lineales debido a la **contigüidad en memoria**, lo que favorece el uso de la memoria caché del procesador y minimiza la sobrecarga de gestión.

2.3. Algoritmo 3: Cálculo de Tarifa

Descripción: Calcula el costo de un viaje basándose en la distancia y el tipo de cliente.

2.3.1. Pseudocódigo

```
1 Funcion CalcularTarifa(distancia, tipoCliente)
2     costoPorKm = 1.0
3     Si tipoCliente == 1 Entonces
4         Retornar distancia * costoPorKm
5     Sino Si tipoCliente == 2 Entonces
6         Retornar distancia * costoPorKm * 0.5
7     Sino
8         Escribir "Error: Tipo no v lido "
9         Retornar -1
10    Fin Si
11 Fin Funcion
```

Listing 3: Cálculo de tarifa por tipo de cliente

2.3.2. Estructura de Datos y Justificación

Se utilizan **variables simples** (float, int). No se requiere una colección de datos ya que el proceso es atómico y no depende de un historial o conjunto de valores.

3. Análisis de Complejidad Temporal

El análisis de complejidad permite predecir el comportamiento del algoritmo ante el crecimiento de la entrada (n).

3.1. Metodología de Cálculo de Complejidad

Para determinar la notación Big O de estos algoritmos, se aplicó el análisis de conteo de operaciones primitivas:

- **Identificación de Bucles:** En el algoritmo de *Palíndromo* y *Segundo Mayor*, identificamos bucles que dependen directamente del tamaño de la entrada (n). Como el código recorre los elementos desde 0 hasta n , se establece una relación lineal.

- **Regla de la Suma:** En el Palíndromo, sumamos el tiempo de limpieza (n) más el tiempo de comparación ($n/2$). Al final, $n + n/2 = 1,5n$, lo que simplificado bajo las reglas de Big O (ignorando constantes) resulta en $O(n)$.
- **Operaciones de Tiempo Constante:** En el cálculo de tarifa, observamos que solo existen asignaciones y comparaciones. Al no haber iteraciones ni recursividad, el número de operaciones no crece aunque la "distancia" sea un número gigante; por tanto, es $O(1)$.
- **Peor de los Casos:** Siempre evaluamos el escenario donde el algoritmo debe trabajar más (por ejemplo, cuando una cadena sí es palíndromo y debe revisarse hasta el final).

Algoritmo	Complejidad	Justificación
Palíndromo	$O(n)$	Recorre la cadena para limpiar y comparar.
Segundo Mayor	$O(n)$	Visita cada elemento del arreglo una vez.
Cálculo Tarifa	$O(1)$	Operaciones constantes, sin bucles.

Cuadro 1: Resumen de Complejidades Temporales

4. Optimización y Límites Teóricos

4.1. ¿Es posible mejorar la complejidad $O(n)$?

Para los algoritmos de búsqueda del segundo mayor y verificación de palíndromo, **no es posible** reducir la complejidad por debajo de $O(n)$.

4.2. Justificación Técnica

1. **Cota Inferior:** Ambos problemas requieren examinar cada dato de entrada al menos una vez para garantizar un resultado correcto. Si se ignorara un solo elemento, no habría certeza de si ese elemento afecta el resultado (por ejemplo, si es un carácter que rompe el palíndromo).
2. **Optimización de Constantes:** Aunque el orden $O(n)$ se mantiene, se puede mejorar la eficiencia práctica. En el palíndromo, se puede evitar crear una nueva cadena procesando la original con dos punteros, reduciendo la complejidad espacial de $O(n)$ a $O(1)$.

5. Conclusión

El estudio de los algoritmos permite no solo resolver problemas, sino hacerlo de la manera más eficiente posible. Mientras que algunos problemas permiten soluciones instantáneas $O(1)$, otros requieren obligatoriamente un tiempo lineal $O(n)$ debido a la naturaleza de la información procesada.